



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой _____ А.В. Юдин

«____» _____ июня
2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки бакалавров 09.04.02 Информационные системы и
технологии

На тему: «Приложение для подготовки к техническим собеседованиям на основе стартапа
ViewTrain»

Обучающийся _____

Сергеев Никита Михайлович

_____ *подпись*

Фамилия, имя, отчество

Шифр 0000000000

Группа ЭФБО-00-22

Руководитель
работы

_____ *подпись*

_____ *Ученая степень, ученое звание,
должность*

_____ *Фамилия, имя, отчество*

Москва 2026 г.

АННОТАЦИЯ

Выпускная квалификационная работа выполнена в формате стартап-проекта в рамках программы «Стартап как диплом» на основе технологического стартапа «ViewTrain».

Предметом исследования для выпускной квалификационной работы выступило написание серверной части приложения для подготовки к техническим собеседованиям на основе стартапа «ViewTrain».

В ходе работы были рассмотрены различные методы для подготовки разработчиков к прохождению интервью при поиске работы в IT-компаниях. Спроектирована и реализована серверная часть приложения ViewTrain на базе FastAPI и PostgreSQL:

- разработаны схема базы данных;
- REST API;
- а также выполнена интеграция с AI-сервисом GigaChat;

Реализована система аутентификации на основе JWT, а также проведены функциональные и интеграционные тестирования: дефекты, которые были найдены – устранены. Сгенерирована документация: к API и по развертыванию приложения.

Ключевые слова: FastAPI, PostgreSQL, JWT, WebSocket, GigaChat, REST API, стриминг, Docker.

Структурно работа состоит из трех глав на 36 страницах и трех приложений на 3 страницах.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ	4
ВВЕДЕНИЕ	5
1 ОПИСАНИЕ СТАРТАП-ПРОЕКТА VIEWTRAIN	7
1.1 Анализ рынка платформ подготовки к техническим собеседованиям	7
1.2 Позиционирование ViewTrain и конкурентный анализ	8
1.3 Целевая аудитория и потребительские сегменты.....	9
1.4 Ценностное предложение и описание продукта.....	10
1.5 Бизнес-модель и вывод на рынок.....	10
1.6 Каналы привлечения и go-to-market.....	11
1.7 Потенциал масштабирования	11
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ПРИЛОЖЕНИЯ	13
2.1 Обзор существующих решений и их серверных архитектур	13
2.2 Анализ требований к backend-части систем с AI-интеграцией.....	14
2.3 Формирование функциональных и нефункциональных требований к серверной части	16
2.4 Постановка задачи на разработку серверной части приложения ViewTrain	18
2.5 Общая архитектура системы ViewTrain.....	18
2.6 Архитектура серверного приложения на базе FastAPI	20
2.7 Проектирование схемы базы данных	21
2.8 Диаграммы системы.....	23
2.8.1 Диаграмма архитектуры взаимодействия клиентской и серверной частей	23
2.8.2 Диаграмма компонентов серверного приложения	24
2.8.3 Диаграммы последовательности основных пользовательских сценариев	25
2.8.4 Диаграмма модели данных (ER-диаграмма).....	26
2.9 Реализация авторизации и управления токенами.....	27
2.10 Разработка REST API для управления пользователями и сессиями.....	28
2.11 WebSocket-сервер для стриминга ответов AI	30
2.12 Интеграция с GigaChat: промпты, обработка потока, типы вопросов.....	31
2.13 Логика управления ходом собеседования на стороне сервера	32
2.14 Тестирование	33
2.14.1 Подход к тестированию: что и зачем проверялось	33
2.14.2 Unit- и интеграционное тестирование	34
2.14.3 Нагрузочное тестирование WebSocket и REST API.....	36
2.14.4 Результаты тестирования и найденные проблемы	36
2.15 Руководство по развертыванию серверной части.....	37
2.15.1 Требования к окружению.....	37

2.15.2 Структура файлов конфигурации	38
2.15.3 Переменные окружения	38
2.15.4 Локальный запуск (development).....	39
2.15.5 Production-развертывание	39
2.15.6 Обновление приложения.....	40
2.16.1 Горизонтальное масштабирование	40
2.16.2 Расширение возможностей промпт-инжиниринга	41
2.16.3 CI/CD и автоматизация	41
3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА.....	42
3.3 Оценка эффективности предложенных решений	42
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45
ПРИЛОЖЕНИЯ	46
Приложение А – Фрагменты исходного кода серверной части.....	47
Приложение Б – Схема базы данных.....	49
Приложение В – Документация API (OpenAPI/Swagger).....	49

ПЕРЕЧЕНЬ ИСПОЛЪЗУЕМЫХ СОКРАЩЕНИЙ

AI – Artificial Intelligence – искусственный интеллект;

API – Application Programming Interface – программный интерфейс приложения;

CRUD – Create Read Update Delete – четыре основные операции при работе с данными на бэкенде

FAANG – компании мирового уровня – Facebook, Apple, Amazon, Netflix, Google;

FK – Foreign Key – внешний ключ в базе данных;

JWT – JSON Web Token – токен для аутентификации;

LLM – Large Language Model – большая языковая модель;

ORM – Object-Relational Mapping – объектно-реляционное отображение;

OT – Operational Transformation – алгоритм для совместного редактирования текста в реальном времени.

REST – Representational State Transfer – архитектурный стиль API;

PK – Primary Key – первичный ключ в базе данных;

PR – Pull Request – запрос на слияние веток;

SPA – Single Page Application – одностраничное приложение;

SQL – Structured Query Language – язык запросов к БД;

TTFT – Time to First Token – время до первого токена ответа;

UUID – Universally Unique Identifier – универсальный уникальный идентификатор;

WebSocket – протокол двусторонней связи через одно TCP-соединение;

БД – база данных;

ВКР – выпускная квалификационная работа.

ВВЕДЕНИЕ

Рынок технических специалистов в сфере информационных технологий в России и за рубежом растет с каждым днем. По данным HH.ru, в 2025 году на одну вакансию разработчика приходилось в среднем 16 резюме, тогда как компании уровня Яндекс, VK и Сбер имеют от трех до шести этапов отбора. Техническое собеседование стало отдельным от умения программировать навыком: кандидат должен уметь объяснять решение вслух, отвечать на уточняющие вопросы, укладываясь в отведенное интервьюером время. Данные требования порождают спрос на инструменты имитации интервью.

Актуальность темы исследования спровоцирована отсутствием доступного инструмента, который заключал бы в себе интерактивный формат собеседования с возможностью практиковаться в любое время без зависимости от другого человека. Существующие платформы – LeetCode, Interviewing.io, Pramp – либо ограничиваются статичными задачами, либо требуют согласования расписания с партнером.

Объектом исследования является серверная часть веб-приложения для подготовки к техническим собеседованиям. Предмет исследования – методы и технологии разработки асинхронного сервиса, обеспечивающего отправку ответов частями в реальном времени, написанного при помощи языка программирования Python с использованием фреймворка FastAPI и интеграции AI-модели.

Цель работы заключена в проектировании и реализации серверной части приложения ViewTrain, обеспечивающей аутентификацию пользователей, управление сессиями собеседований, выдачу ответов AI-интервьюера через WebSocket и хранение истории диалогов в реляционной базе данных.

Для достижения цели поставлены следующие задачи:

1. провести анализ рынка платформ подготовки к техническим собеседованиям и изучить архитектуры существующих решений;

2. сформулировать функциональные и нефункциональные требования к разрабатываемой серверной части
3. спроектировать архитектуру сервиса и модели базы данных;
4. реализовать REST API, WebSocket-сервер и интеграцию с API GigaChat;
5. провести функциональное и интеграционное тестирование;
6. написать документацию к API и развертыванию приложения.

Практическая значимость работы состоит в доступности и полезности разработанного сервиса, который сразу готов к интеграции с фронтендом и стабильно и без аномалий готовит нуждающегося человека к прохождению технической секции собеседования.

1 ОПИСАНИЕ СТАРТАП-ПРОЕКТА VIEWTRAIN

1.1 Анализ рынка платформ подготовки к техническим собеседованиям

Рынок инструментов подготовки к техническим собеседованиям начал формироваться в конце 2010-х годов вместе с тем, как росли технологические компании, где алгоритмическая, теоретическая секция задач и структурированные интервью стали основой найма. По данным Market Research Intellect, объем рынка платформ для подготовки к техническим собеседованиям составил 500 миллионов долларов в 2024 году и по прогнозам достигнет 1,2 миллиарда долларов к 2033 году. Основные приложения для подготовки к собеседованиям делятся на два направления: платформы с базами задач и сервисы, которые имитируют живое интервью с базовыми вопросами по теме.

Крупнейший представитель первого лагеря – LeetCode. По данным компании, к 2024 году на платформе зарегистрировано более 20 миллионов пользователей. Монетизация построена на freemium-подписке: на бесплатном тарифе доступна лишь часть задач, нет AI анализатора кода, платный Premium (35 долларов в месяц) – полный каталог, разборы решений и вопросы по определенным компаниям из FAANG. Еще есть платформа – HackerRank, которая работает схожим образом, но в первую очередь ориентирована на компании, которые используют платформу при найме сотрудников в штат. Это создает два направления выручки – от компаний и от людей, которые хотят устроиться в эти же компании.

Сервисы второго лагеря изнутри устроены совсем по-другому. Pramp предлагает парные тестовые интервью: два пользователя меняются ролями – сначала один выступает интервьюером, потом другой. Доступ к приложению – бесплатный, заработок идет только за счет компаний, которые сами активно используют приложение. Interviewing.io работает с реальными специалистами

из Google, Amazon, Meta – они проводят интервью добровольно или за деньги от самой платформы, что делает прохождение максимально приближенным к настоящему собеседованию.

Таблица 1.1 – Сравнительный анализ платформ подготовки к техническим собеседованиям

Платформа	Формат практики	Обратная связь	Доступность	AI-компонент	Стоимость
LeetCode	Задачи, автопроверка кода	Прошел / не прошел тесты	24/7	AI-ассистент разбора (с 2024)	Freemium, \$35/мес.
HackerRank	Задачи, тесты для компаний	Автопроверка	24/7	Нет	Freemium
Pramp	Парное mock-интервью	Оценка партнера	По расписанию, ожидание 2–4 дня	Нет	Бесплатно
Interviewing.io	Интервью с реальным специалистом FAANG	Детальная от эксперта	По расписанию	Нет	\$225–300 за сессию
CoderPad	Совместный редактор кода	От интервьюера	По расписанию	Нет	Для компаний
ViewTrain	AI-диалог, стриминг ответов	Структурированная после каждого ответа	24/7, без ожидания	GigaChat – основа системы	Freemium, 290 руб./мес.

1.2 Позиционирование ViewTrain и конкурентный анализ

Слабое место Pramp и Interviewing.io – зависимость от другого человека. Пользователи платформ в отзывах на Product Hunt и Trustpilot сообщают, что среднее время ожидания пары занимает от нескольких часов до нескольких дней. Для жителей регионов с малым числом активных участников и для тех, кто хочет практиковаться ночью или в выходные дни, это ограничение особенно ощутимо.

ViewTrain закрывает эту проблему: AI-интервьюер доступен в любой момент, не привязан к определенному расписанию и не зависит от часового пояса пользователя. Приложение не позиционирует себя, как замена реальным

тестовым интервью – скорее как инструмент ежедневной разминки перед тем, как идти на Pramp, а потом уже на настоящее собеседование. Целевая аудитория – разработчики уровня junior и middle, готовящиеся к переходу в крупную технологическую компанию: FAANG, Яндекс, Сбер, VK.

ViewTrain является компромиссом там, где одни компании выбирают либо собеседование с настоящим человеком по расписанию, либо недостаточный опыт интервью и просто решение задач.

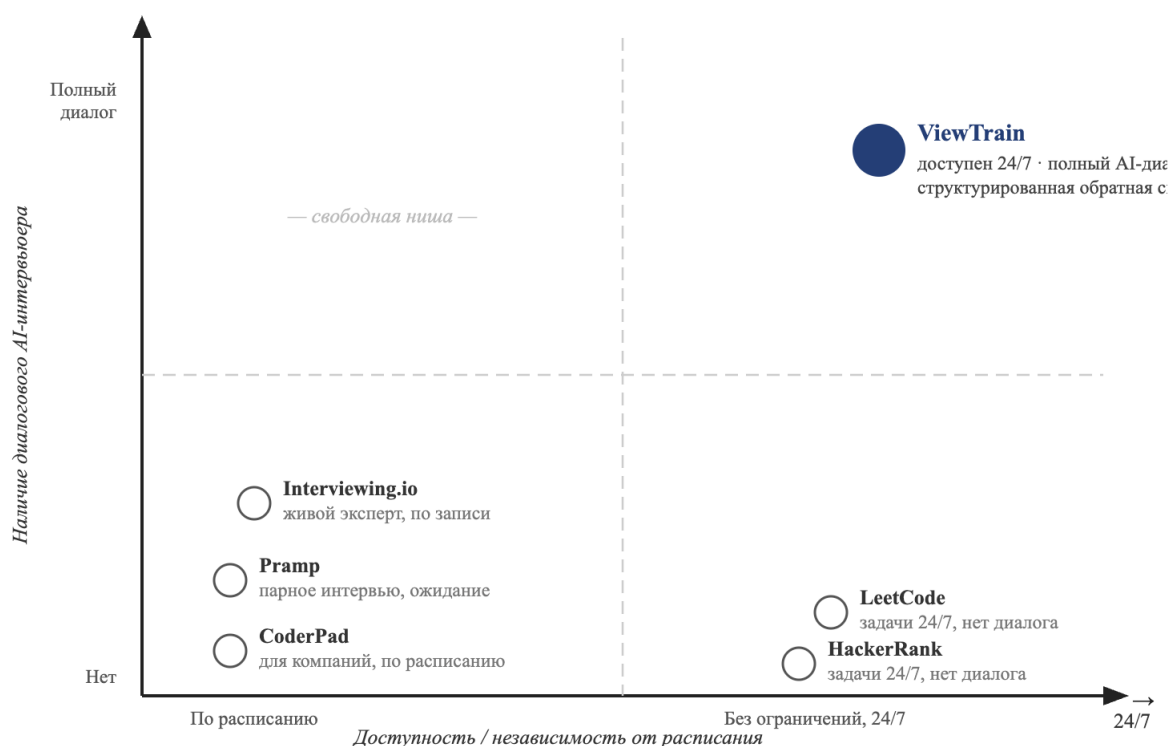


Рисунок 1.1 – Позиционирование ViewTrain на рынке платформ подготовки к техническим собеседованиям

1.3 Целевая аудитория и потребительские сегменты

ViewTrain закрывает три сегмента аудитории.

Основной – Junior и Middle разработчики, которые готовятся к собеседованиям в IT-компаниях. Менторы – дорогие, Pramp требует определенного расписания и знания английского. Постоянной практики взять нигде. Соответственно платить готовы, так как разница в зарплате между грейдами того стоит.

Второй – студенты старших курсов перед своими первыми стажировками. Опыта интервью у них нет, формат собеседований пугает. Денег у них меньше, чем у уже работающих разработчиков, но при доступной цене готовы пользоваться долго.

Третий – самый перспективный: образовательные платформы и корпоративные университеты – можно лицензировать ViewTrain как модуль подготовки.

1.4 Ценностное предложение и описание продукта

ViewTrain симулирует техническое собеседование с AI интервьюером. Практиковаться можно в любой момент, без ожидания партнера и без живого ментора, к которому надо записываться.

Есть три формата: алгоритмические задачи, system design и поведенческое интервью. Каждый из этих форматов имеет по три уровня сложности – junior, middle и senior.

Внутри – GigaChat: задает вопросы, разбирает ответы и дает обратную связь.

Backend на FastAPI, PostgreSQL и WebSocket, ответы выдаются частями, соответственно приложение не зависает даже на длинных ответах от ИИ.

От конкурентов приложение отличает четыре вещи: доступен 24/7, работает на русском языке, сложность подстраивается под уровень пользователя и дешевле, чем живой ментор.

1.5 Бизнес-модель и вывод на рынок

ViewTrain – freemium приложение. Бесплатный тариф – несколько сессий в месяц, хватит для того, чтобы пользователь мог попробовать формат. Чтобы пользователь подготовился серьезнее – Есть модель Premium: без лимита сессий, все типы интервью, расширенный разбор ответов.

В корпоративном тарифе будет использоваться оплата за одно место. IT-школы и корпоративные университеты будут получать админку для того, чтобы управлять группами и получать отчет по активности.

Для того чтобы получить первую аудиторию, будут использоваться Habr и Telegram-каналы для разработчиков.

1.6 Каналы привлечения и go-to-market

Есть три канала, которые расставлены по приоритету по стоимости привлечения.

Первый — это контент в профессиональных сообществах. Он главный на старте. Habr, Telegram-каналы про разработку и карьеру – разбор реальных задач, сравнение различных подходов подготовки и истории пользователей. Аудитория там уже ищет инструменты, поэтому надо оказаться рядом в нужный момент.

Второе – это партнерство с айти-школами, корпоративными университетами и различными учебными центрами. Сразу две функции: привлечение пользователей и старт B2B.

И третья – это реферальная программа, последняя, когда сформирована база активных платежей пользователей.

1.7 Потенциал масштабирования

ViewTrade масштабируется в трех направлениях, и для этого не нужно переписывать технологическую основу.

Первое – продуктовое: новые форматы не из разработческой специальности: системные аналитики, дизайнеры и проект-менеджеры. Также можно добавить более глубокие форматы для опытных, например, расширенный системный дизайн.

Второе направление — это географическое. Начало с русскоязычной аудитории России и СНГ, потом Казахстан, Беларусь, Армения. Потом — локализация на английский язык.

И третье направление — это техническое. FastAPI и PostgreSQL горизонтально масштабируются, подробнее в разделе 2.16. Пользователей можно прибавлять без изменения бэкэнда.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ПРИЛОЖЕНИЯ

2.1 Обзор существующих решений и их серверных архитектур

У каждого конкурента – построена своя архитектура, которую скопировать не получится, в связи с приватностью исходного кода, но можно использовать как ориентир при проектировании.

LeetCode изнутри устроен как набор микросервисов. Задачи выполняются в изолированной среде через Judge0 – опенсорс приложение для запуска кода в режиме песочницы. Основной REST API построен на TypeScript; фронтенд обменивается с ним по HTTPS. LeetCode обрабатывает тысячи запросов на компиляцию кода, каждый – несколько секунд в своем окружении. Главная задача – раздать очередь так, чтобы пользователи не ждали других и не тратили ресурсы за счет других компиляций.

Pramp решает другую задачу – соединить двух пользователей для живого интервью. Аудио и видео идут через WebRTC, а редактор кода синхронизируется по WebSocket. Нагрузка на backend не вычислительная, сервер помогает установить peer-to-peer соединение, потом отходит и данные идут напрямую между браузерами.

Ни одна из архитектур конкурентов не подходит без доработок. Judge0 – слишком избыточен, так как код выполнять не нужно. WebRTC – тоже не нужен, потому что ИИ не требуется голос или видео. OT – тоже не нужен, у приложения один участник – пользователь. Что нужно: WebSocket – GigaChat отдает текст частями, пользователь видит ответ по мере его генерации, а не ждет полное время, пока текст будет написан. REST API для работы с сессиями и пользователями, PostgreSQL, для хранения информации.

Общее у всех приложений – JWT или сессионные куки, реляционная база данных. ViewTrain использует ту же схему, но проще. Вместо микросервисов – один сервис, вместо того, чтобы использовать распределенное хранилище – используется одна база данных.

LeetCode	Pramp	ViewTrain
<i>Язык / фреймворк</i>		
TypeScript + Django REST	Node.js	Python · FastAPI
<i>База данных</i>		
MySQL + Redis	PostgreSQL	PostgreSQL (5 таблиц)
<i>Протокол соединения</i>		
HTTPS + WebSocket (Channels)	WebRTC + WebSocket	HTTPS + WSS (стриминг)
<i>Ключевая задача</i>		
Fair scheduling тысяч компиляций кода	P2P-соединение между пользователями	Стриминг AI-ответов по чанкам через WS
<i>Архитектура</i>		
Микросервисы	Монолит	Монолит · 1 БД

Рисунок 2.1 – Сравнение серверных архитектур конкурирующих платформ

2.2 Анализ требований к backend-части систем с AI-интеграцией

LLM добавляет серверной части специфичные требования, которых в обычных CRUD-приложениях нет.

Первое – задержки. GigaChat не отвечает мгновенно: до первого токена уходит 1-2 секунды, полный разбор задачи занимает 8–20 секунд. Держать пользователя в ожидании – нельзя, так как может показаться, что что-то зависло. Решение одно – backend получает ответ частями и сразу же отображает его пользователю, не дожидаясь конца генерации ответа от LLM. Это требует асинхронного сервера и постоянного соединения, поэтому – используются WebSocket.

Второе – управление контекстом. GigaChat не хранит всю историю переписки в себе, каждый запрос идет вместе со всеми предыдущими сообщениями в виде списка messages. 10-15 пар «вопрос-ответ» – уже несколько тысяч токенов потрачено. Backend хранит историю чата, сжимает из нее контекст и следит за лимитом.

Третье – обработка ошибок. API GigaChat может вернуть 429 (превышен лимит запросов) или 503 (сервис временно недоступен), или вообще оборвать соединение на половине запроса. Чтобы пользователь понимал, что произошло, а не просто обрывало его сессию – нужно грамотно обрабатывать ошибки и выдавать их пользователю. Backend различает два типа сбоев: временные – лечатся перезапросами к API с задержкой, постоянные – нет.

Четвертое – безопасность ключей. Секреты хранятся в переменных среды на сервере и клиент не имеет доступа к ним. Клиент напрямую не обращается к API GigaChat, только через сервер. Это определяет структура, вся логика приложения: промпты и обработка ответов на стороне сервера.

Таблица 2.1 – Требования к backend-части систем с AI-интеграцией и технические решения

Требование	Проблема	Техническое решение
Управление задержками	Время до первой буквы от GigaChat – 1–2 с, полный ответ – 8–20 с. Блокировать пользователя нельзя: воспринимается как зависание	WebSocket-стриминг: backend пересылает каждый чанк клиенту немедленно, не дожидаясь конца генерации
Хранение контекста диалога	GigaChat не хранит историю – каждый запрос требует полной переписки в виде списка messages	История сессии хранится в таблице messages PostgreSQL; загружается и передается в API перед каждым запросом
Обработка ошибок API	GigaChat может вернуть 429, 503 или оборвать соединение посередине стриминга	Разделение временных (retry с задержкой) и постоянных сбоев; клиент получает сообщение без закрытия WebSocket
Безопасность ключей	API-ключ GigaChat не должен попадать к клиенту ни при каких условиях	Ключ хранится в переменных окружения сервера; клиент обращается только к backend-прокси, никогда напрямую к GigaChat

2.3 Формирование функциональных и нефункциональных требований к серверной части

Из анализа вышло два набора требования к backend: функциональные – что система делает и нефункциональные – как хорошо она это делает.

Таблица 2.2 – Нефункциональные требования к серверной части ViewTrain

Идентификатор	Описание
ФТ-01 Регистрация	Регистрация по email и паролю: email проверяется на уникальность, при успехе сервер возвращает пару токенов.
ФТ-02 Аутентификация	Вход по email и паролю: совпали данные - access-токен на 15 минут и refresh-токен на 30 дней.
ФТ-03 Обновление токена	Пришел refresh-токен – получил новую пару. Старый сразу отзывается.
ФТ-04 Создание сессии	Пользователь выбирает тип интервью (алгоритмы, system design) и сложность – сервер создает сессию и берет первый вопрос из базы.
ФТ-05 Проведение сессии	Сообщение приходит через WebSocket – сервер собирает промпт с историей, запрашивает GigaChat и отдает ответ клиенту по мере генерации.
ФТ-06 История сессий	API возвращает список завершенных и активных сессий пользователя
ФТ-07 Детали сессии	Запрос по id сессии – в ответ полная переписка от первого сообщения до последнего.
ФТ-08 Статистика	Статистика по пользователю: сколько сессий, каких типов и как долго в среднем.

Таблица 2.3 – Функциональные требования к серверной части ViewTrain

Идентификатор	Критерий приемлемости
НФТ-01 Задержка REST API	При 100 одновременных пользователях 95% запросов укладываются в 200 мс.
НФТ-02 TTFT стриминга	Пользователь отправил сообщение – первая часть ответа появляется на экране в течение 1,5 секунды.
НФТ-03 Конкурентность	100 активных WebSocket-соединений – REST API при этом замедляется не больше чем на 30%.
НФТ-04 Безопасность	Пароли хранятся как bcrypt-хэш, JWT подписан HS256 API-ключ GigaChat клиент не видит
НФТ-05 Восстанавливаемость	Ошибка GigaChat не рвет сессию и не закрывает WebSocket - клиент не видит сообщение об ошибке и может продолжить
НФТ-06 Развертывание	Вся серверная инфраструктура запускается одной командой docker compose up в любой среде с Docker compose

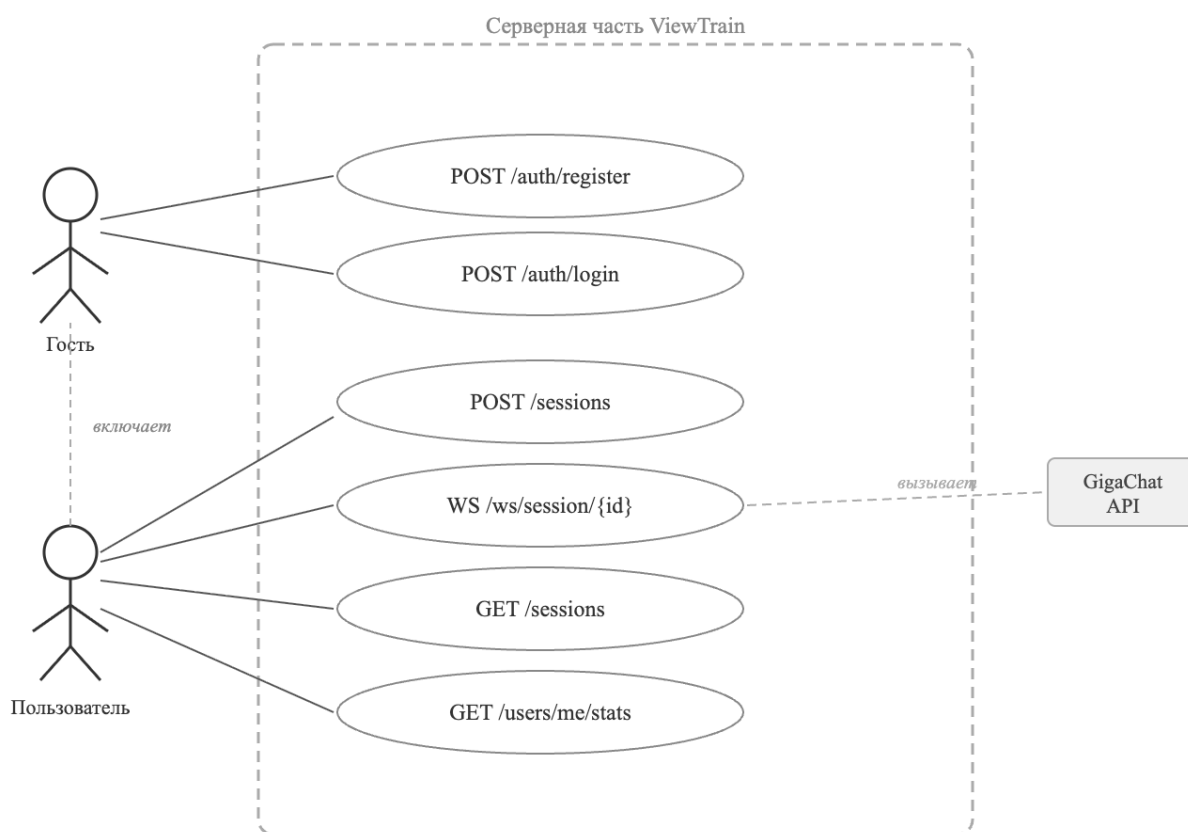


Рисунок 2.2 – Диаграмма вариантов использования серверной части ViewTrain

2.4 Постановка задачи на разработку серверной части приложения ViewTrain

Из анализа сложилась конкретная задача. Backend сервис для ViewTrain на Python и FastAPI. Единственная точка входа для клиента и посредник между ним и GigaChat.

Что входит в scope: FastAPI приложение, схема ORM модели и миграции, REST API для пользователей и сессий, WebSocket сервер для стриминга, JWT аутентификация, интеграция с GigaChat API, деплой через Docker Compose. Что не входит в обязанности: клиентская часть приложения - ее разрабатывает другой человек команды и CI/CD.

FastAPI выбран не случайно. Нативная поддержка асинхронной разработки нужен для блокировки на ожидании GigaChat, чтобы остальные запросы встали в очередь. Автоматическая генерация документации OpenAPI – меньше ручной работы. Pydantic валидирует входные данные из коробки. FastAPI быстрее Django Rest Framework в 2-3 раза.

Для работы с PostgreSQL выбрана связка - SQLAlchemy, asyncpg и Alembic. Это является неким стандартом для написания проектов на FastAPI. SQLAlchemy работает с базой данных через объектную модель, asyncpg не блокирует event loop, Alembic автоматически генерирует миграции.

На выходе получается: backend с REST API и WebSocket, Swagger документация, тесты с покрытием более 75% и Docker Compose файл для поднятия сервера с базой одной командой.

2.5 Общая архитектура системы ViewTrain

ViewTrain состоит из трех компонентов: клиент, backend-сервис и API GigaChat. Напрямую клиент с GigaChat не общаются – все идет через backend.

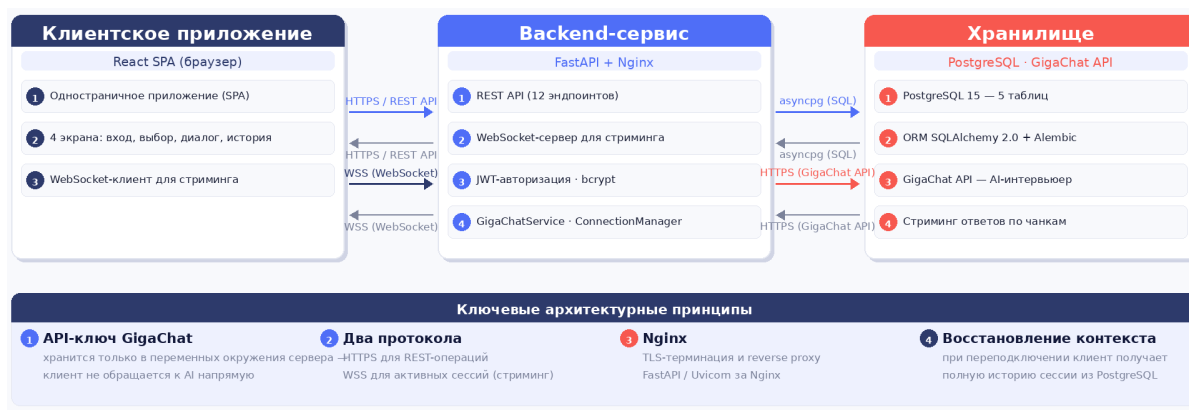


Рисунок 2.3 – Общая архитектура системы ViewTrain

Клиент работает по протоколу HTTPS и WSS. Первый для REST API – авторизация, история и профиль, а второй – для активных сессий. Backend идет в PostgreSQL по Docker сети без выхода наружу, а в GigaChat по HTTPS через публичный доступ. Соответственно клиент не общается с внешним API напрямую, а делает запросы через серверную прослойку.

Из этого вытекает несколько плюсов. Ключ API GigaChat не выходит с сервера, тем самым не попадает в браузер. История сессий хранится в базе данных на сервере. Если оборвалась сеть, то после переподключения сохраняется весь контекст. Rate limits и квоты токенов контролируются в одном месте, а не раскидываются по клиентам.

Nginx проксирует весь трафик и отвечает за TLS: HTTP идет на FastAPI, WebSocket тоже туда же. FastAPI в свою очередь на Uvicorn с внешним трафиком, TLS напрямую не работает благодаря Nginx.

2.6 Архитектура серверного приложения на базе FastAPI

Приложение построено на слоистой архитектуре – четыре слоя, где каждый отвечает за свою зону. Роутеры принимают запросы и передают их сервисам. В сервисах хранится вся бизнес логика, про базу данных и HTTP не знают ничего. Репозитории работают с базой данных через библиотеку SQLAlchemy. Модели описывают структуру таблиц базы данных.

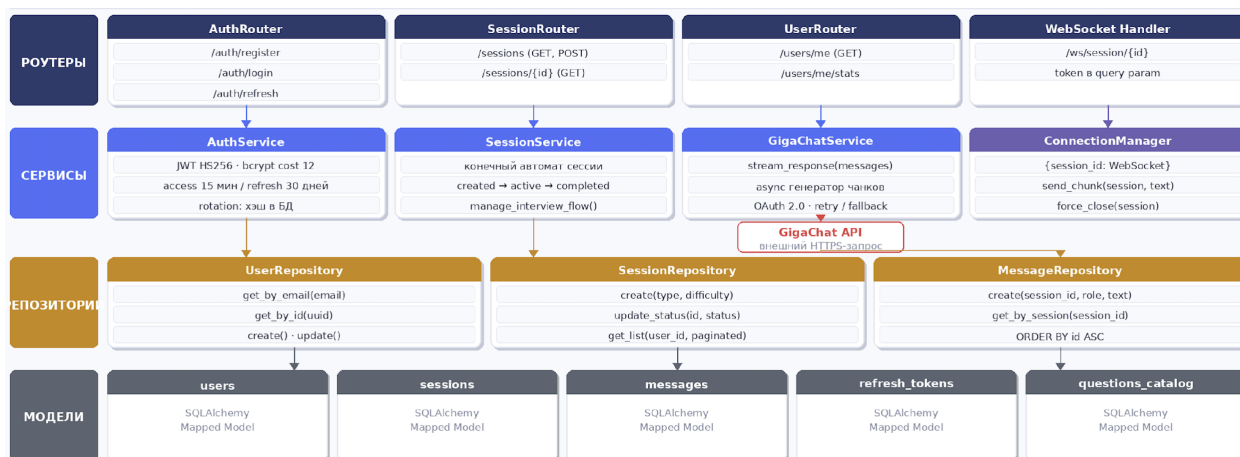


Рисунок 2.4 – Диаграмма компонентов серверного приложения

Структура директорий проекта организована по принципу модулей:

Листинг 2.1 – Структура директорий серверного приложения

```
app/
├── api/
│   ├── auth.py           # эндпоинты авторизации
│   ├── sessions.py       # эндпоинты управления сессиями
│   ├── users.py          # эндпоинты профиля пользователя
│   └── ws.py             # WebSocket-обработчик
├── core/
│   ├── config.py         # настройки через pydantic-settings
│   └── security.py       # JWT-утилиты, bcrypt
├── db/
│   ├── models.py         # SQLAlchemy-модели
│   └── session.py        # фабрика async-соединений
├── repositories/
│   ├── user_repo.py
│   ├── session_repo.py
│   └── message_repo.py
├── services/
│   ├── auth_service.py
│   ├── session_service.py
│   └── gigachat_service.py
├── schemas/              # Pydantic-схемы запросов и ответов
└── main.py              # точка входа, регистрация роутеров
```

FastAPI объявляет зависимости через Depends. Функция достает JWT токен из заголовка авторизации и возвращает User – подключается одной аннотацией в коде. Так как токены обрабатываются в одном месте, middleware для каждой ручки писать не надо.

База данных работает через asynсpg в связке с SQLAlchemy и Alembic для миграций таблиц. SQLAlchemy нужен для того, чтобы была возможность создавать таблицы и работать с ними средствами Python. Asynсpg не блокирует event loop при запросах к базе. Один клиент ждет ответа от базы, а event loop переключился на другого. Несколько WebSocket соединений в одном потоке работают конкурентно.

Конфигурация приложения находится в модели Pydantic в файле config. Значения читаются из переменных окружения: все легко тестируется. В Docker Compose переменные передаются через .env файл.

2.7 Проектирование схемы базы данных

Перед тем, как формировать таблицы был составлен список регулярных запросов. Что вошло в этот список: проверка почты на уникальность при

регистрации, поиск пользователя по id для валидации, поиск refresh токена по хэшу, создание и обновление записи сессии, добавление сообщения в историю и выборка сессии с фильтрацией по статусу и типу.

Схема – пять таблиц. Таблица users: id (UUID, PK), email (varchar(255), unique, not null), username (varchar(100), not null), hashed_password (varchar(255), not null), created_at (timestamp, default now()), is_active (boolean, default true). UUID выбран вместо serial/int, потому что он генерируется на клиенте без обращения к базе данных и не светит id системе.

Таблица refresh_tokens – для обновления access-токенов. Поля: id (UUID), user_id (FK -> users.id, CASCADE DELETE), token_hash (varchar(64), unique – SHA-256-хэш токена), expires_at (timestamp), revoked (boolean, default false), created_at. В базе хранится хэш, а не токен: утечет база – получат хэш, а не токены.

Таблица sessions – данные сессий. id (UUID), user_id (FK -> users.id), interview_type (algorithms, system_design, behavioral), difficulty (junior, middle, senior), status (created, active, completed, abandoned), started_at (timestamp, nullable), ended_at (timestamp, nullable), created_at.

Таблица messages – история каждой сессий. id (bigserial – порядок вставки гарантирован), session_id (FK → sessions.id, CASCADE DELETE), role (user, assistant), content (text), created_at (timestamp). Сессия удалась – вся ее история тоже. Сообщений без нее не сохраняются.

Таблица questions – банк вопросов для старта сессии. id (UUID), interview_type, difficulty, content (text). Вопрос при создании сессии копируется в messages первым сообщением. Каталог потом можно менять – история старых сессий не изменится.

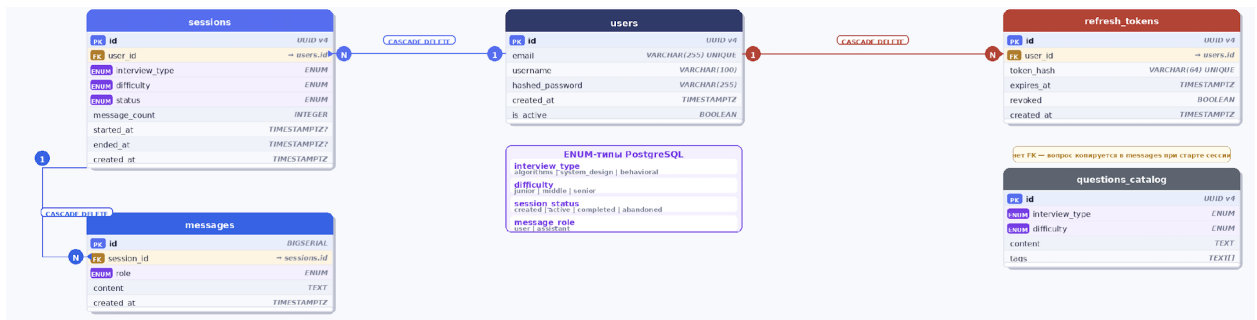


Рисунок 2.5 – ER-диаграмма базы данных ViewTrain

Помимо автоматических индексов по первичным ключам, вручную добавлены три. `sessions(user_id, status)` – выборка активных сессий пользователя, самый частый запрос. `messages(session_id, created_at)` – выборка переписки по хронологии. `refresh_tokens(token_hash)` – поиск по хэшу при обновлении токена. PostgreSQL индексы для FK-полей не создает сам, в отличие от MySQL – нужно самому контролировать.

2.8 Диаграммы системы

2.8.1 Диаграмма архитектуры взаимодействия клиентской и серверной частей

На рисунке 2.6 – архитектура взаимодействия клиентской и серверной части. Клиент держит два соединения с сервером: HTTPS для обычных операций и постоянное WSS для активных сессий собеседования.

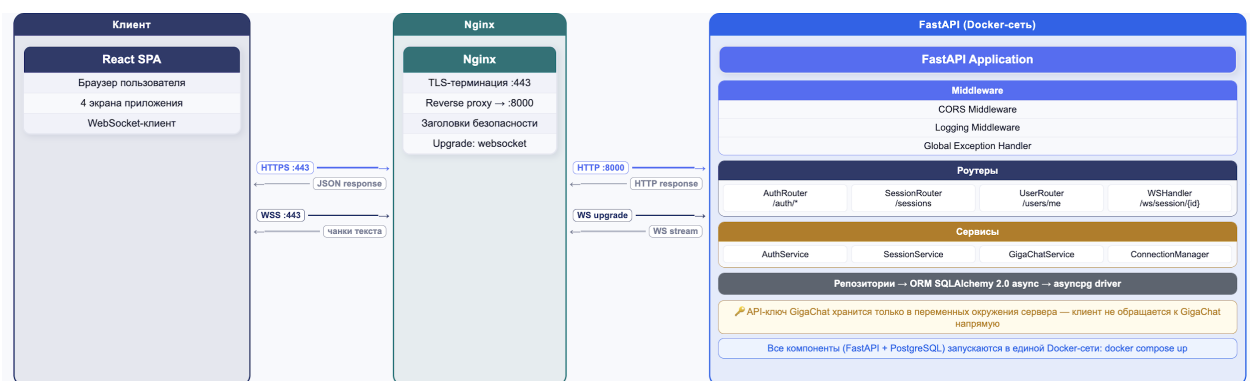


Рисунок 2.6 – Диаграмма архитектуры взаимодействия клиентской и серверной частей

Nginx снимает TLS и прогоняет трафик на FastAPI внутри докера с заголовками безопасности по пути.

Все запросы в FastAPI идут через middleware. CORS-middleware проверяет origin и решает пускать или нет. Логирование пишет путь и время обработки. Глобальный handler ошибок перехватывает исключения и если что-то упало, то возвращается 500.

2.8.2 Диаграмма компонентов серверного приложения

На рисунке 2.7 – внутренняя структура backend-приложения. Компоненты не знают друг о друге напрямую: каждый отвечает за свое и общается с остальными через фиксированные интерфейсы. Это упрощает тестирование и замену отдельных частей без переписывания всего остального.

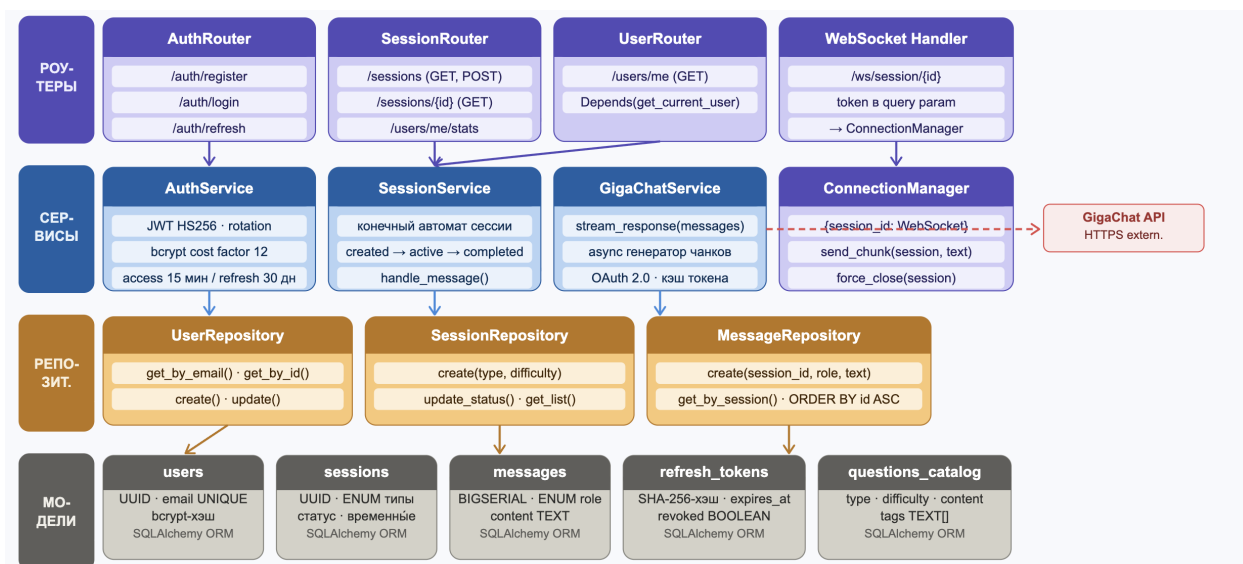


Рисунок 2.7 – Внутренняя структура компонентов backend-приложения

AuthRouter принимает запросы /auth/* и отправляет их в AuthService. AuthService знает о хэшировании паролей, создании и валидации токенов, но ничего не знает об роутерах и базе данных. UserRepository предоставляет сервису четыре метода: get_by_email(), get_by_id(), create(), update(). Такое же строится для SessionRouter -> SessionService -> SessionRepository и MessageRepository.

GigaChatService стоит особняком: он не является ни роутером, ни репозиторием. Это асинхронная обертка над AsyncClient для общения с API

GigaChat. Его использует SessionService – получает в ответ асинхронный генератор частей ответа GigaChat.

2.8.3 Диаграммы последовательности основных пользовательских сценариев

Стоит разобрать подробнее авторизацию и ход в активной сессии собеседования.

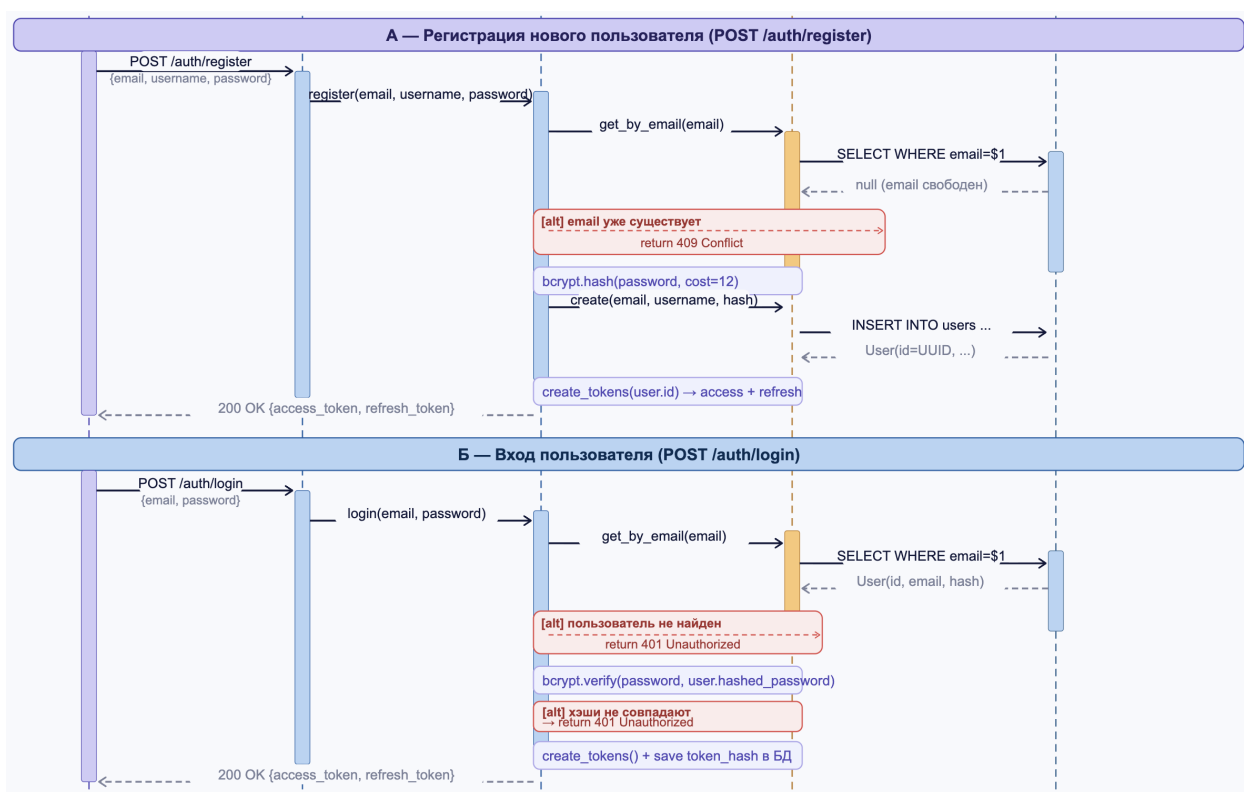


Рисунок 2.8 – Диаграмма последовательности: регистрация и вход

При регистрации клиент отправляет POST запрос на регистрацию с телом запроса {email, username, password}. Сервис проверяет через UserRepository, есть ли пользователь с такой почтой в базе данных. Если email уже используется, то возвращается 409 ошибка. Если почта свободна, пароль шифруется, пользователь добавляется в базу данных, и выдается пара токенов – еще раз входить после регистрации не надо. При логине сервер находит пользователя через почту, сравнивает хэши пароля и при совпадении создает новую пару токенов.

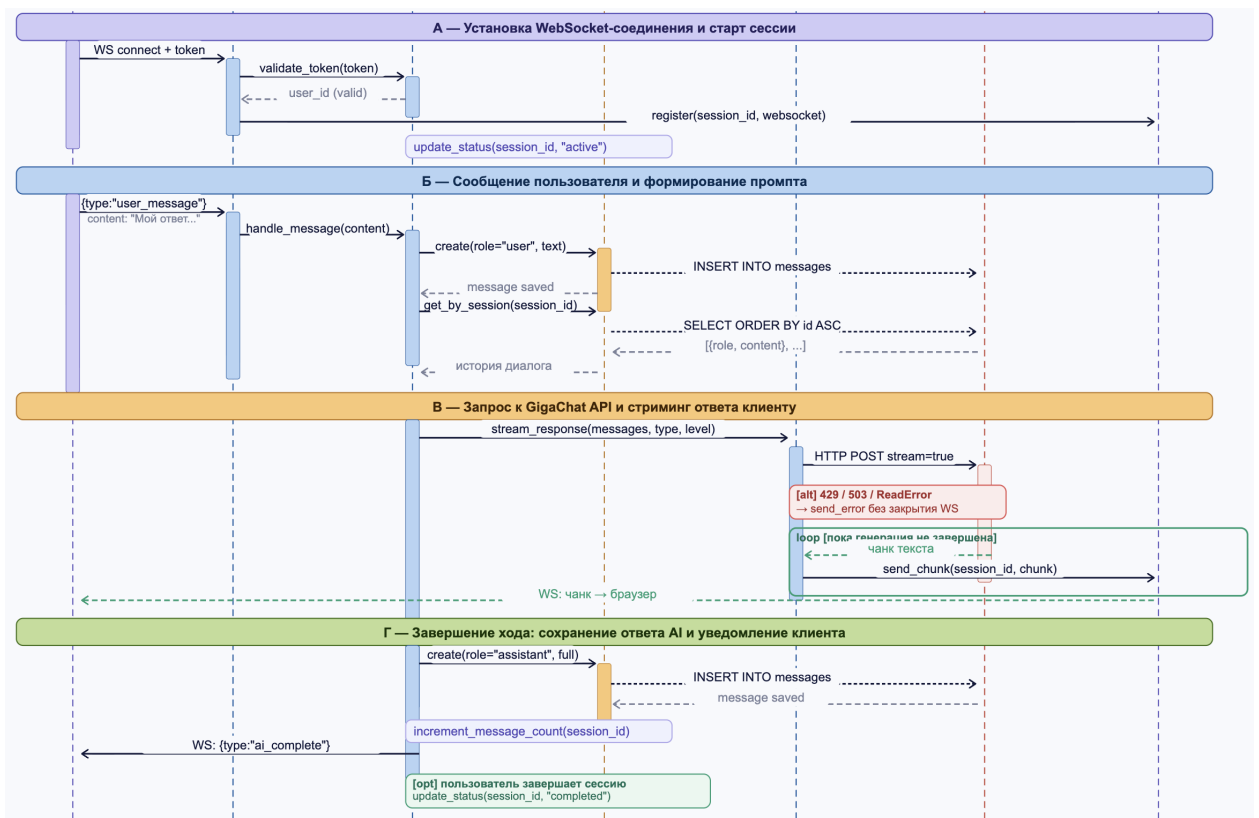


Рисунок 2.9 – Диаграмма последовательности: ход в сессии собеседования

2.8.4 Диаграмма модели данных (ER-диаграмма)

На рисунке 2.10 – ER-диаграмма на физическом уровне: имена таблиц, атрибуты и типы данных так, как в схеме БД.

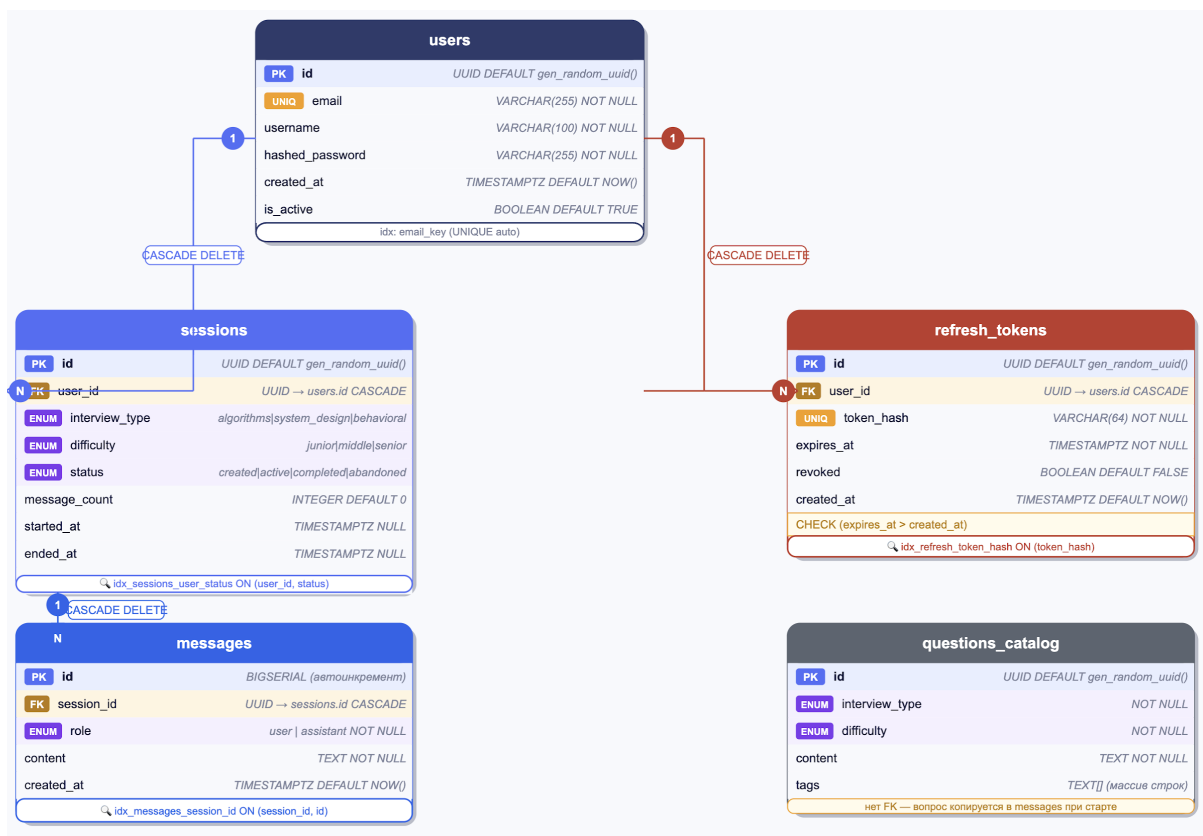


Рисунок 2.10 – ER-диаграмма базы данных ViewTrain (физический уровень)

Связь users -> sessions – один ко многим (1:N): один пользователь может иметь любое количество сессий. Связь sessions -> messages – тоже 1:N. Связь users -> refresh_tokens – 1:N с удалением по очереди. Таблица questions_catalog не имеет прямых FK-связей с другими таблицами: вопрос при старте сессии копируется как первое сообщение в messages. Это дает редактировать каталог без изменения сессий, которые уже завершены.

2.9 Реализация авторизации и управления токенами

Access-токен не хранится в базе данных, тем самым нагрузка на базу меньше и легче масштабировать. Подпись JWT – HS256, ключ 32 байта из переменной SECRET_KEY в переменных окружения.

Access-токен живет 15 минут. Если он будет жить слишком долго – то украденный токен остается работать. Если слишком мало – то клиент все время должен получать новый. 15 минут, обычно, стандарт в приложениях, которые не имеют особых требований к безопасности.

Refresh-токен – случайная строка из 32 байт, которая живет 30 дней. В базе данных только хэш этой строки. При обновлении клиент шлет токен в теле запроса, сервер делает хэш и ищет запись – три условия: существует, не отозванный и не истек. Прошло – новая пара токенов, старый токен отзывается. Access-токен хранить незачем: через 15 минут его действие закончится.

Листинг 2.2 – Зависимость FastAPI для получения текущего пользователя

```

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    try:
        payload = jwt.decode(token, settings.SECRET_KEY,
            algorithms=["HS256"])
        user_id = UUID(payload["sub"])
    except (JWTError, ValueError, KeyError):
        raise HTTPException(status_code=401, detail="Invalid token")
    user = await user_repo.get_by_id(db, user_id)
    if not user or not user.is_active:
        raise HTTPException(status_code=401, detail="User not found")
    return user

```

Зависимость подключается одной аннотацией. FastAPI сам добавляет заголовок авторизации и в Swagger упрощается ручное тестирование.

2.10 Разработка REST API для управления пользователями и сессиями

REST API – URL называет ресурс, а не действие и используем HTTP методы по назначению. Базовый URL – /api/v1. Версия в префиксе при помощи fastapi-versioning. Так явнее и кэш браузера не ломается. Документация автоматическая: Swagger и ReDoc.

Таблица 2.4 – Эндпоинты REST API ViewTrain

Метод	Путь	Описание	Auth
POST	/auth/register	Регистрация нового пользователя	Нет
POST	/auth/login	Вход, получение пары токенов	Нет
POST	/auth/refresh	Обновление access-токена по refresh	Refresh
POST	/auth/logout	Отзыв refresh-токена	Да

GET	/users/me	Профиль текущего пользователя	Да
PATCH	/users/me	Обновление имени пользователя	Да
GET	/users/me/stats	Агрегированная статистика по сессиям	Да
POST	/sessions	Создание новой сессии	Да
GET	/sessions	Список сессий с пагинацией и фильтрацией	Да
GET	/sessions/{id}	Детали сессии с полной историей сообщений	Да
PATCH	/sessions/{id}	Обновление статуса сессии (завершить, покинуть)	Да

Формат ответов стандартный: успех – массив, ошибка – мапа {“detail”: “описание ошибки”}. Коды стандартные, 2xx – успех, 401 – проблема с токеном, 404 – нет объекта, 422 – не проходит валидация Pydantic.

Пагинация работает через параметры запроса: ?limit=20&offset=0. Это просто в реализации и достаточно для текущих объемов данных. Дополнительные фильтры работают так же через параметры запроса, которые передаются репозиторию, а он в свою очередь добавляет WHERE-условия к SQL-запросу.

2.11 WebSocket-сервер для стриминга ответов AI

WebSocket-эндпоинт располагается по адресу `/ws/session/{session_id}` и принимает токен как `query`-параметр. Параметр запроса вместо заголовка авторизации используется из-за того, что браузер не дает устанавливать свои заголовки WebSocket соединению, а только URL.

При создании соединения сервер сразу смотрит на токен: если он неправильный или истекший, соединение будет закрыто с кодом 4001 (Custom Application Error). Если токен корректен, соединение создается в `ConnectionManager`, который держит таблицу уже активных соединений:

Листинг 2.3 – ConnectionManager для управления WebSocket-соединениями

```
class ConnectionManager:
    def __init__(self):
        self.active: dict[UUID, WebSocket] = {}

    async def connect(self, session_id: UUID, ws: WebSocket):
        if session_id in self.active:
            await self.active[session_id].close(code=4002)
        await ws.accept()
        self.active[session_id] = ws

    async def send_chunk(self, session_id: UUID, chunk: str):
        if ws := self.active.get(session_id):
            await ws.send_json({"type": "ai_chunk", "content": chunk})

    async def disconnect(self, session_id: UUID):
        self.active.pop(session_id, None)
```

При новом подключении сервер смотрит, нет ли уже активного соединения. Если есть, то закрывает старое с кодом 4002 и регистрирует новое. То есть несколько соединений не остается.

Протокол строго типизирован. Клиент всегда отправляет одно и то же в формате: `{type: "user message", content: "..."}.` Сервер может ответить тремя способами – часть текущего ответа, сигнал завершения с полным текстом для истории или описание ошибки. Фронтенд на каждый реагирует по-своему.

Если соединение разрывается, пока выводится ответ, FastAPI выдает исключение. Обработчик ловит его и записывает в логи. Сессия остается активной и продолжит работу при следующем подключении, поэтому история диалога не теряется.

2.12 Интеграция с GigaChat: промпты, обработка потока, типы вопросов

GigaChat API принимает запросы в формате, который совместим с API ChatGPT: POST /chat/completions с полями model, messages и stream. Аутентификация двухэтапная: сначала по client_id и client_secret получается OAuth токен на 30 минут, потом он идет в заголовок Authorization: Bearer для запросов к /chat/completions. GigaChatService кэширует токен в памяти и обновляет за 60 секунд до конца действия, из-за этого активные сессии не разрываются.

Промпты – отдельная история. Для каждого из интервью свой промпт для того, чтобы модель лучше подстраивалась под конкретное собеседование и под конкретного пользователя. Интервью по алгоритмам: под определенный уровень пользователя – определенные задачи, оценка сложности по времени и по памяти. System Design: проектирование систем с обоснованием выбора конкретного стека, могут быть уточнения, но не прямые подсказки. Поведенческие вопросы – конкретные примеры с прошлых мест работы, если такие есть.

Листинг 2.4 – Асинхронный генератор вывода ответов GigaChat

```
async def stream_response(
    self, messages: list[dict]
) -> AsyncGenerator[str, None]:
    headers = {"Authorization": f"Bearer {await self._get_token()}"}
    payload = {"model": "GigaChat", "messages": messages, "stream": True}
    try:
        async with httpx.AsyncClient(timeout=30) as client:
            async with client.stream("POST", GIGACHAT_URL,
                                    json=payload, headers=headers) as
response:
                async for line in response.aiter_lines():
                    if line.startswith("data: ") and line != "data: [DONE]":
                        data = json.loads(line[6:])
                        content = data["choices"][0]["delta"].get("content",
""")
                            if content:
                                yield content
    except httpx.ReadError as e:
        raise GigaChatStreamError(f"Stream interrupted: {e}") from e
```

2.13 Логика управления ходом собеседования на стороне сервера

Сессия собеседования имеет четыре состояния. **CREATED**: сессия создана после POST запроса, WebSocket соединение еще не открыто. **ACTIVE**: соединение есть, диалог начат. **COMPLETED**: пользователь завершил собеседование сам или достиг лимита сообщений. **ABANDONED**: сессия была активной, но из-за произошел таймаут.

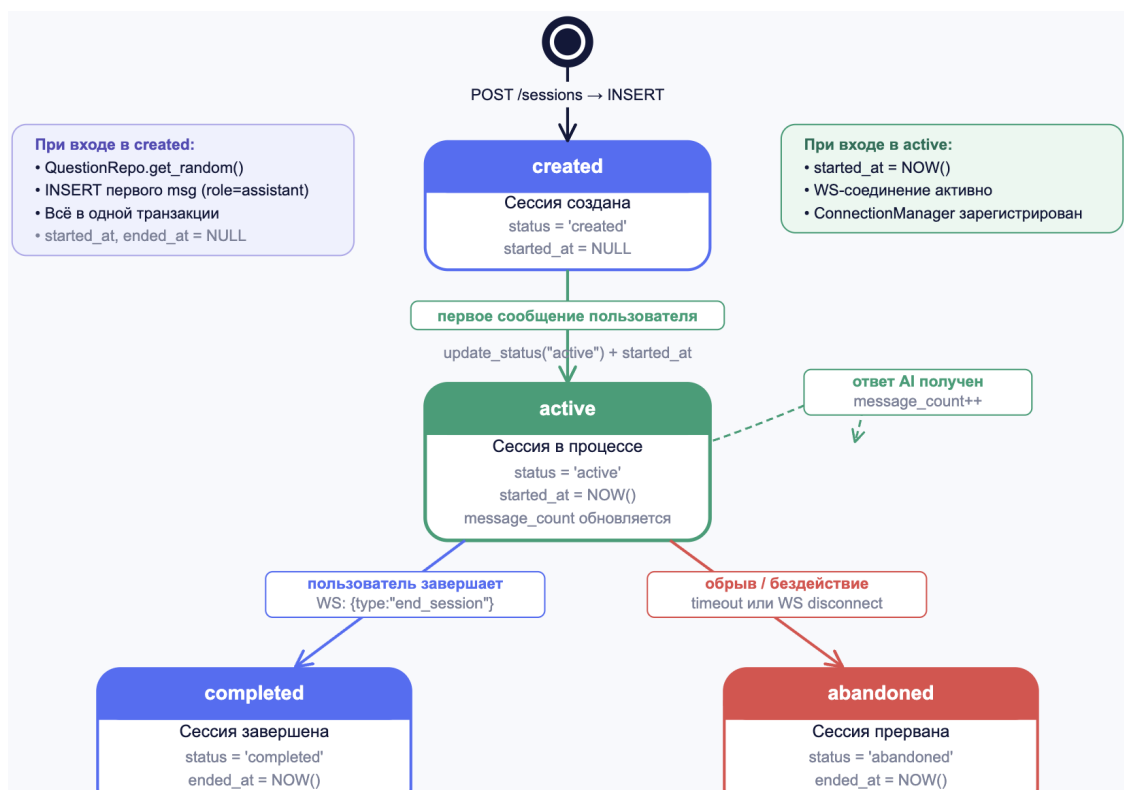


Рисунок 2.11 – Диаграмма состояний сессии собеседования

При создании сессии сервер выбирает вопрос из базы вопросов по типу интервью и сложности. При повторных сессиях одного типа вопросы не повторяются. Вопрос записывается в messages и возвращается клиенту в ответе. При WebSocket-подключении SessionService проверяет статус. **CREATED** переходит в **ACTIVE**, после этого статус не меняется и шлет клиенту историю сообщений, то есть после переподключения история остается.

Сессия завершается тремя способами. Либо пользователь сам завершает сессию и клиент отправляет COMPLETED. Пользователь достиг лимита сообщений MAS_SESSION_MESSAGES и также переводит в COMPLETED. Либо по таймауту, пользователь превысил SESSION_TIMEOUT, клиент переводит в ABANDONED.

2.14 Тестирование

2.14.1 Подход к тестированию: что и зачем проверялось

Тестирование разбито на три уровня, где каждый закрывает свою зону.

Unit-тесты проверяют сервисы в изоляции от базы данных и внешних API. В score: логика JWT – подпись, время истечения, отклонение не рабочих и просроченных токенов, написание промптов под разные интервью, изменение статусов сессий. Репозитории, GigaChat и ConnectionManager – моковые данные.

Интеграционные тесты идут от HTTP запроса до ответа, с реальной тестовой базой данных. Каждый тест работает внутри транзакции, которая до завершения откатывается, соответственно база не меняется. GigaChat – мок, реальные запросы дорогие, запросы через асинхронный клиент httpx. HTTP сервера нет.

Нагрузочные тесты – Locust: сценарии на Python, WebSocket через сторонний клиент, интерфейс с различными графиками.

Общая структура тестовых директорий:

Листинг 2.5 – Структура директории тестов

```
tests/
├── conftest.py          # фикстуры: подключение к тестовой БД, async клиент
├── unit/
│   ├── test_auth_service.py
│   ├── test_session_service.py
│   ├── test_gigachat_service.py
│   └── test_session_fsm.py
├── integration/
│   ├── test_auth_api.py
│   ├── test_sessions_api.py
│   └── test_websocket.py
└── load/
    ├── locustfile.py
    └── ws_scenario.py
```

2.14.2 Unit- и интеграционное тестирование

Unit-тесты написаны с использованием pytest. Есть три теста JWT: правильный токен проходит валидацию, все работает, закончившийся токен вызывает HTTPException с ошибкой 401, токен не принимается из-за неправильной подписи. Пример теста истекшего токена:

Листинг 2.6 – Unit-тест: отклонение истекшего JWT

```
def test_expired_token_raises_401():
    expired_token = create_access_token(
        user_id=uuid4(),
        expires_delta=timedelta(seconds=-1)
    )
    with pytest.raises(HTTPException) as exc_info:
        decode_access_token(expired_token)
    assert exc_info.value.status_code == 401
```

Интеграционные тесты покрывают все пути пользователя. Тест полного юзерфлоу авторизации проверяет регистрацию, повторная регистрация с той же почтой, вход, обновление токена и выход:

Листинг 2.7 – Интеграционный тест: цикл авторизации

```
async def test_auth_flow(client: AsyncClient):
    # 1. Регистрация
    r = await client.post("/auth/register", json={
        "email": "test@example.com",
        "username": "testuser",
        "password": "SecurePass123!"
    })
    assert r.status_code == 201
    access = r.json()["access_token"]

    # 2. Повторная регистрация с тем же email
    r2 = await client.post("/auth/register", json={
        "email": "test@example.com",
        "username": "other",
        "password": "SecurePass123!"
    })
    assert r2.status_code == 409

    # 3. Запрос с токеном
    r3 = await client.get("/users/me",
        headers={"Authorization": f"Bearer {access}"})
    assert r3.status_code == 200
    assert r3.json()["email"] == "test@example.com"
```

В итоге покрытие кода составляет в сервисном слое 78%, авторизация 91%, репозитории – 63%. Общее покрытие – 75%, что соответствует показателю, заданному в требованиях.

2.14.3 Нагрузочное тестирование WebSocket и REST API

Нагрузочное тестирование проводилось в два этапа. На первом тестировались REST-эндпоинты с нарастающей нагрузкой от 10 до 200 виртуальных пользователей, потом WebSocket с симуляцией активных сессий. Среда: Apple MacBook Pro M1 Pro, 16 ГБ оперативной памяти, Docker Desktop 4.28.0, PostgreSQL 15 в контейнере, FastAPI с Uvicorn в 4 рабочих процессах.

Сценарий REST: каждый виртуальный пользователь входит в аккаунт, потом в цикле делает GET /sessions каждые 2 секунды. Результаты приведены в таблице 3.1.

Таблица 3.1 – Результаты REST-нагрузочного тестирования

Пользователей	rps	p50 latency	p95 latency	p99 latency	Ошибок
10	18	31 мс	67 мс	104 мс	0%
50	82	38 мс	124 мс	189 мс	0%
100	143	52 мс	198 мс	341 мс	0%
150	161	89 мс	412 мс	870 мс	0,3%
200	158	134 мс	891 мс	2 140 мс	1,8%

При 150+ пользователей p95 вышел за 200 мс, проблема – пул соединений по умолчанию был 10, и запросы блокировались в очереди. пул был увеличен, p95 вернулся в норму, 0% ошибок.

2.14.4 Результаты тестирования и найденные проблемы

В тестировании нашлось несколько проблем.

Первая – N+1 запросов в GET /sessions. 20 сессий в ответе, репозиторий делал 1 запрос за списком сессий и 20 за их количеством в отдельных сообщениях, получается 21 запрос вместо одного.

Вторая – гонка состояний при быстром переключении по WebSocket. Клиент открывал новое соединение раньше, чем сервер закрывал старое. Ответ GigaChat из-за этого дублировался. Исправлено было тем, что

новое соединение открывается только после того, когда старое принудительно закрывалось. (листинг 2.3).

Третья – обрыв стриминга при падении HTTP-соединения с GigaChat. Если GigaChat обрывает соединение во время генерации – `asyncio` выдает `httpx.ReadError`. Если нет обработки, то просто прилетает неожиданное исключение в `WebSocket`. `FastAPI` закрывает соединение с кодом 1011. Клиент видит разрыв, сессия зависает в активном состоянии.

Четвертая проблема – не проверялся владелец сессии. GET запрос на `/session/{id}` отдавал сессию по `id` кому угодно, надо было просто знать `UUID`. То есть любой пользователь, который имел аккаунт в приложении мог прочесть чужую переписку. Эта ошибка была исправлена простым запросом в репозиторий: `WHERE user_id = :current_user_id`. Если какое то несовпадение - то выдавало 404 ошибку, вместо 403, так просто вообще не видно другому пользователю существует сессия или нет.

2.15 Руководство по развертыванию серверной части

Серверная часть приложения `ViewTrain` запакована в `Docker` контейнеры и запускается одной командой `docker compose up`. То есть благодаря этой команде поднимается `backend` часть и база данных одновременно, проводятся миграции, после чего серверная часть полностью готова к использованию. В данном разделе описаны все шаги, которые необходимы для запуска приложения на любой системе с `Docker`.

2.15.1 Требования к окружению

Для того, чтобы развернуть `ViewTrain` нужна операционная система, с установленной на ней `Docker` и `Docker Compose`. Минимум 2 ГБ оперативной памяти и минимум 10 ГБ постоянной. Доступ компьютера в интернет и `GigaChat API`.

Для production среды также нужен домен и сертификат к нему, чтобы приложение работало по https.

2.15.2 Структура файлов конфигурации

Корневой каталог проекта содержит следующие конфигурационные файлы:

Листинг 2.8 – структура файлов конфигурации

```
viewtrain-backend/
├── docker-compose.yml      # оркестрация контейнеров
├── docker-compose.prod.yml # production среда
├── Dockerfile              # образ FastAPI-приложения
├── nginx/
│   └── nginx.conf          # конфигурация reverse proxy
├── .env.example            # пример переменных окружения
├── alembic/
│   └── versions/           # миграции базы данных
```

2.15.3 Переменные окружения

Перед запуском приложения нужно создать файл .env на основе .env.example и заполнить обязательные переменные. Все секретные данные передаются в приложение только через переменные среды и не хранятся ни в коде приложения, ни в самом репозитории. Обязательные переменные для того, чтобы приложение запустилось – DATABASE_URL – строка подключения к базе данных PostgreSQL, SECRET_KEY – случайная строка для подписи JWT, GIGACHAT_API – данные для получения доступа к GigaChat API.

Опциональные переменные среды – POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD – из них собирается DATABASE_URL, но также можно указать вручную. ACCESS_TOKEN_EXP_MINUTES – сколько живет access-токен в минутах, по умолчанию 15, REFRESH_TOKEN_EXP_DAYS – срок жизни refresh-токена в днях, по умолчанию 30.

2.15.4 Локальный запуск (development)

Для того, чтобы запустить приложение локально для тестирования нужно выполнить следующие команды:

Листинг 2.9 – локальный запуск приложения

```
git clone https://github.com/nlx9s/viewtrain.git
cd viewtrain
cp .env.example .env
docker compose up --build
```

Этим командами пользователь клонирует репозиторий себе в систему, после переходит в папку с проектом и там заполняет файл с переменными среды, после чего запускает приложение и получает доступ к сервису по адресу <http://localhost:8000>. Документация к API доступна по адресу <http://localhost:8000/docs> или <http://localhost:8000/redoc>.

2.15.5 Production-развертывание

Для того, чтобы запустить приложение в production-среде используется отдельный файл для развертывания, который переопределяет некоторые параметры: отключает документацию к API, чтобы пользователи извне не могли получить доступ к ней, включает автоматический перезапуск контейнеров при сбоях, также ограничивает ресурсы контейнеров. Для запуска нужно выполнить следующие команды:

Листинг 2.10 – развертывание приложения на сервере

```
docker compose -f docker-compose.prod.yml \
    up -d --build
docker compose ps
docker compose logs app --tail=50
```

Этим командами пользователь поднимает контейнеры с production конфигурацией, проверяет состояние контейнеров и проверяет логи приложения, запустилось или нет.

Nginx слушает порты 80 и 443, HTTP-трафик на порту 80 автоматически перенаправляется на 443 порт на HTTPS. Для получения SSL сертификата для HTTPS: `certbot --nginx -d (домен)`. После этого бот для получения сертификата автоматически добавит в Nginx параметры TLS.

2.15.6 Обновление приложения

Обновление приложения без его остановки выполняется так: новый образ собирается параллельно с контейнером, который уже работает, после чего он заменяется с помощью `docker compose up -d --no-deps --build app`. Потом применяются миграции, и предыдущая версия контейнера останавливается и удаляется.

2.16 Направления дальнейшего развития серверной части

В процессе разработки и тестирования backend части приложения ViewTrain было найдено несколько направлений, после реализации которых повысится производительность и масштабируемость системы.

2.16.1 Горизонтальное масштабирование

Нынешняя архитектура рассчитана на запуск в пределах одного сервера. При росте количества пользователей свыше 500 одновременных сессий будет нужно горизонтальное масштабирование. Надо будет вынести ConnectionManager из памяти процесса в Redis. Благодаря этому можно будет запускать несколько процессов приложения одним общим балансировщиком нагрузки.

При переходе на такую архитектуру не надо изменять frontend или базу данных, нужно только изменение ConnectionManager.

2.16.2 Расширение возможностей промпт-инжиниринга

В нынешней реализации используются фиксированные промпты в системе для трех типов интервью. Возможные улучшения: версионирование промптов в базе данных, адаптация сложности вопросов на основе ответов определенного пользователя, то есть персонализация промпта под пользователя. Поддержка дополнительных типов интервью, например, системный дизайн с различными диаграммами, SQL.

Разделение логики промптов в отдельный сервис уже заложено в архитектуре через GigaChatService, соответственно, после разделения не нужно будет изменять роутеры или репозитории.

2.16.3 CI/CD и автоматизация

Сейчас деплой вручную выполняется командой `docker compose up`. Для команды, которая работает над продуктом в проде, нужен полноценный ci/cd пайплайн для того, чтобы все изменения автоматически проходили тесты и проверялись на покрытие при каждом PR. Также нужна автоматическая сборка приложения и поднятие docker контейнера при слиянии ветки в main. Чтобы создать пайплайн в GitHub Actions нужно около 15 часов работы. Это сократит время деплоя в 5 раз, с 15 минут ручных действий до 3 минут автоматического процесса.

3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА

Экономическая часть: сколько стоит разработка backend части приложения, какой доход ожидается при запуске и когда проект выйдет в плюс.

3.1 Затраты на разработку

Разработка серверной части сервиса велась в рамках обучения на третьем курсе. Чтобы оценить стоимость похожего продукта используется метод трудоемкости – по реально потраченному времени и средней ставке Python разработчика в Москве.

Проектирование архитектуры – 40 часов, реализация модуля авторизации – 24 часа, разработка REST API – 32 часа, реализация WebSocket сервера – 28 часов, интеграция с API GigaChat – 24 часа, написание тестов – 32 часа. Итого 180 часов.

Средняя рыночная ставка Python разработчика Junior уровня в Москве – от 1000 до 1500 рублей в час по данным Habr Карьеры. Берем среднее значение 1250 рублей в час и получаем $1250 * 180 = 225000$ рублей.

3.2 Инфраструктурные расходы

Для того, чтобы запустить приложение на коммерцию требуется следующая инфраструктура: виртуальный сервер – около 5000 рублей в месяц, база данных PostgreSQL около 3000 рублей в месяц, GigaChat API – около 2000 рублей в месяц при 1000 активных пользователей и сессией по 15 минут в среднем, домен и ssl сертификат – 1500 рублей в год, около 120 рублей в месяц.

3.3 Оценка эффективности предложенных решений

Технические решения проекта напрямую влияют на экономику.

Монолит вместо микросервисов – дешевле на старте. Одна инфраструктура, где нет различных расходов на взаимодействие между сервисами.

Архитектура в виде слоев дает возможность выделить различные компоненты в отдельные сервисы при росте нагрузки.

WebSocket вместо обычного HTTP – лучше. Первые слова выдаются через 1-2 секунды, а не после 20 секунд ожидания, когда весь текст будет выдан пользователю. Меньше зависаний в процессе, соответственно выше удержание в платном тарифе.

Асинхронный FastAPI не блокируется на длинных запросах к GigaChat. Одновременных пользователей на том же сервере может быть больше, чем при синхронной реализации – нет ожидания между пользователями.

Тесты снижают стоимость, так как каждое изменение проверяется автоматически и ошибки не доходят до production.

ЗАКЛЮЧЕНИЕ

В рамках Выпускной Квалификационной Работы был спроектирован и реализован backend ViewTrain – приложения для подготовки к техническим собеседованиям с AI собеседующим на базе GigaChat.

В теоретической части был проведен анализ рынка и архитектур конкурентов проекта: LeetCode, Pramp. Ни одна из платформ не дает живое интервью без привязки к расписанию другого человека. Также были выявлены требования к backend с привязкой LLM: стриминг для управления задержками, история диалога, обработка ошибок внешнего API.

На аналитическом этапе – архитектура FastAPI приложения и схема базы данных. Выбор FastAPI обоснован тем, что это быстрый современный фреймворк, но который так же имеет большую поддержку со стороны сообщества. Автоматическая генерация документации по стандарту OpenAPI, валидация Pydantic.

На практике реализовано: JWT с refresh-токенами, REST-API из 25+ эндпойнтов, Web-Socket сервер для показа ответа, интеграция GigaChat с промптами под разные форматы интервью. Три разных формы тестов: юнит тесты, покрытие кода более чем 75%, интеграционные тесты с моковой базой данных и нагрузочные тесты с 200 одновременных виртуальных пользователей. Все дефекты устранены.

Все поставленные задачи выполнены, цель работы достигнута. Разработанный сервис готов к использованию в качестве backend приложения для подготовки к техническим собеседованиям и запускается одной командой `docker compose up` в любой среде с Docker.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Электронные ресурсы:

1. FastAPI Documentation. – Режим доступа: <https://fastapi.tiangolo.com> (дата обращения: 01.03.2025).
2. PostgreSQL 17 Documentation. – Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 10.03.2025).
3. GigaChat API Reference. Сбер / Sber AI. – Режим доступа: <https://developers.sber.ru/docs/ru/gigachat/api/overview> (дата обращения: 15.03.2025).
4. TechEmpower Framework Benchmarks. – Режим доступа: <https://www.techempower.com/benchmarks> (дата обращения: 20.02.2025).
5. JWT Best Practices. Auth0 Blog. – Режим доступа: <https://auth0.com/blog/a-look-at-the-latest-draft-for-jwt-bcp> (дата обращения: 05.03.2025).
6. Microservices. – Режим доступа: <https://microservices.io/> (дата обращения: 01.03.2025).
7. HH.ru. – Режим доступа: <https://sense-it.ru/mediacenter/zarplaty-it-spetsialistov-v-rf-osnovnye-trendy-iii-kvartala-2025-goda> (дата обращения: 10.03.2026).
8. Market Research. – Режим доступа: <https://www.marketresearchintellect.com/product/mock-interview-platforms-market/> (дата обращения: 10.03.2026).

ПРИЛОЖЕНИЯ

Приложение А – Фрагменты исходного кода серверной части

Приложение Б – Схема базы данных

Приложение В – Документация API (OpenAPI/Swagger)

Приложение А

Фрагменты исходного кода серверной части

A.1 – Модели базы данных (app/db/models.py)

```
import uuid
from sqlalchemy import String, Boolean, DateTime, Text, Enum, ForeignKey,
BigInteger
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import DeclarativeBase, relationship, mapped_column,
Mapped
from datetime import datetime

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    email: Mapped[str] = mapped_column(String(255), unique=True,
nullable=False, index=True)
    username: Mapped[str] = mapped_column(String(100), nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=datetime.utcnow)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    sessions: Mapped[list['Session']] = relationship(back_populates='user',
cascade='all, delete-orphan')

class Session(Base):
    __tablename__ = 'sessions'
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey('users.id', ondelete='CASCADE'))
    interview_type: Mapped[str] =
mapped_column(Enum('algorithms', 'system_design', 'behavioral',
name='interview_type'))
    difficulty: Mapped[str] = mapped_column(Enum('junior', 'middle', 'senior',
name='difficulty'))
    status: Mapped[str] =
mapped_column(Enum('created', 'active', 'completed', 'abandoned',
name='session_status'), default='created')
    started_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    ended_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=datetime.utcnow)
    user: Mapped['User'] = relationship(back_populates='sessions')
    messages: Mapped[list['Message']] =
relationship(back_populates='session', cascade='all, delete-orphan')

class Message(Base):
    __tablename__ = 'messages'
    id: Mapped[int] = mapped_column(BigInteger, primary_key=True,
autoincrement=True)
    session_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey('sessions.id', ondelete='CASCADE'))
    role: Mapped[str] = mapped_column(Enum('user', 'assistant',
name='message_role'))
```

```

        content: Mapped[str] = mapped_column(Text, nullable=False)
        created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=datetime.utcnow)
        session: Mapped['Session'] = relationship(back_populates='messages')

```

A.2 – Поимер аёморузайуу (app/api/auth.py)

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db
from app.services.auth_service import AuthService
from app.schemas.auth import RegisterRequest, LoginRequest, TokenResponse

router = APIRouter(prefix='/auth', tags=['auth'])

@router.post('/register', response_model=TokenResponse, status_code=201)
async def register(body: RegisterRequest, db: AsyncSession =
Depends(get_db)):
    service = AuthService(db)
    return await service.register(body.email, body.username, body.password)

@router.post('/login', response_model=TokenResponse)
async def login(body: LoginRequest, db: AsyncSession = Depends(get_db)):
    service = AuthService(db)
    return await service.login(body.email, body.password)

@router.post('/refresh', response_model=TokenResponse)
async def refresh(refresh_token: str, db: AsyncSession = Depends(get_db)):
    service = AuthService(db)
    return await service.refresh(refresh_token)

```

Приложение Б

Схема базы данных

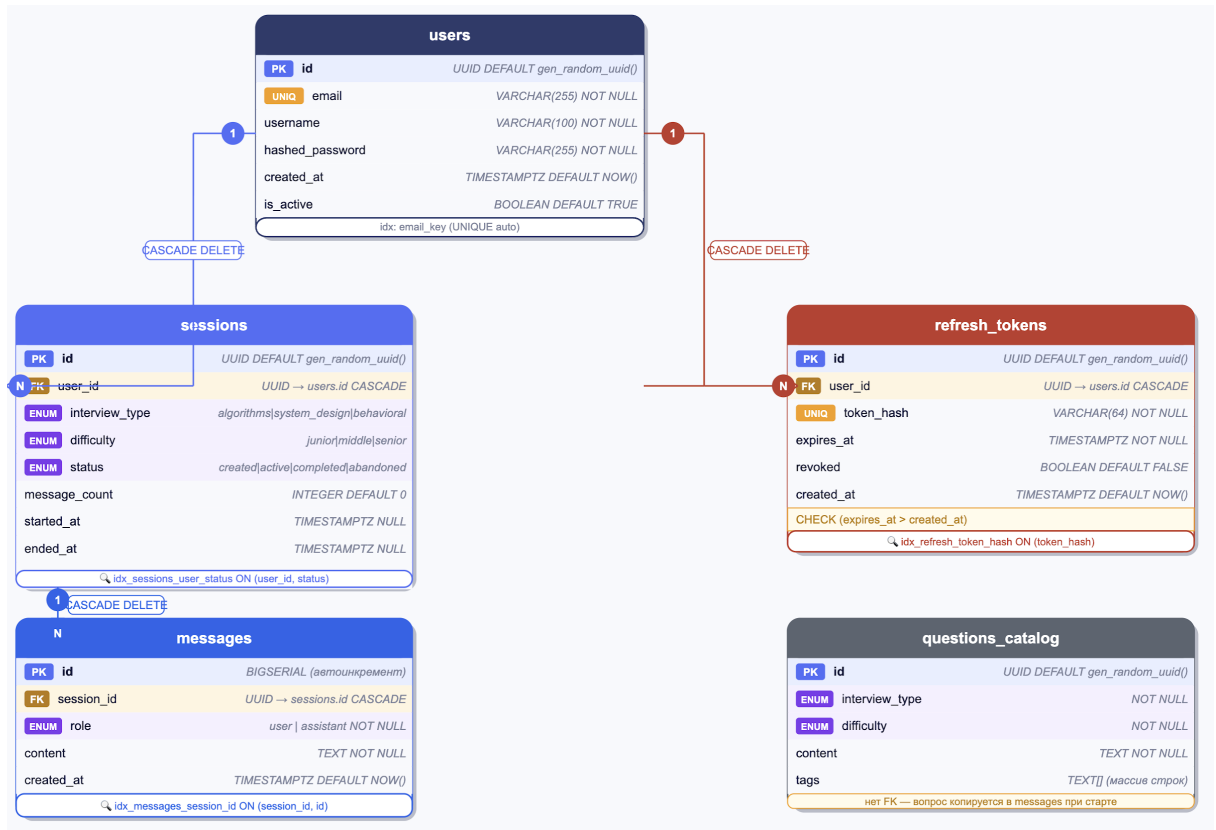


Рисунок Б.1 – Физическая ER-диаграмма базы данных ViewTrain

Приложение В

Документация API (OpenAPI/Swagger)

Auth			^
POST	/api/v1/auth/register	Register User	v
POST	/api/v1/auth/login/	Auth User	v
POST	/api/v1/auth/refresh	ФТ-03: Обновление токена	v

Рисунок В.1 – Swagger UI: endpoints авторизации

Sessions			^
POST	/api/v1/sessions	ФТ-04: Создание сессии	v
GET	/api/v1/sessions	ФТ-06: Список сессий	v
GET	/api/v1/sessions/{session_id}	ФТ-07: Детали сессии	v

Рисунок В.2 – Swagger UI: endpoints управления сессиями